Ashen Road

CRC Cards | EC-334 Game Design and Development

Hassan Ayoub (BCSF23M004) Hafiz Own (BCSF23M018) Abdul Basit (BCSF23M022)

IState

Interface. Implemented by: PlayerState, EnemyState, BossState

Responsibilities	Collaborators
• Declare Enter, Tick, FixedTick and Exit hooks	–

StateMachine

Plain C# class

Responsibilities	Collaborators
• Know current and previous state • Change state (Exit old, Enter new) • Ignore re-entry of the active state unless forced • Forward Tick / FixedTick to current state • Return to the previous state • Announce every change (StateChanged)	IState

GameState / GameStateRules

Enum + static transition table

Responsibilities	Collaborators
• List the game states (Boot, MainMenu, Loading, Playing, Paused, GameOver, LevelComplete, Ending, GameFinished) • Know which transitions are legal • Say which states are gameplay	–

GameEvents

Static event channel (Observer)

Responsibilities	Collaborators
• Hold the global game events (state, level, player, checkpoint, front, boss, hits, sounds, effects, hints) • Provide safe Raise methods • Clear all listeners between tests • Keep no game state	–

GameManager

Singleton

Responsibilities	Collaborators
• Persist one instance across scenes • Know game state, level, checkpoint, death cause, last result, deaths • Validate state transitions and announce them • Start, retry, restart or advance levels; go to menu • Pause and resume (time scale, input map) • Complete a level: save best time, unlock next level • React to player death and checkpoints • Begin the ending and finish the game	GameEvents GameStateRules SceneLoader SaveSystem LevelCatalog InputReader ScreenFader LevelResult

GameBootstrap

Static

Responsibilities	Collaborators
• Create the persistent Systems prefab before the first scene loads • Make Play work from any scene	GameManager

SceneLoader

Static

Responsibilities	Collaborators
• Load the menu or level scene asynchronously • Know loading progress • Know which scene is active	–

SaveSystem

Plain C# class. Uses: ISaveStore (PlayerPrefsStore, MemorySaveStore)

Responsibilities	Collaborators
• Know highest unlocked level; unlock levels • Record and report best times • Know music and effects volume • Know whether the game was finished • Reset progress, keeping volumes	ISaveStore

ObjectPool

Generic (Object Pool)

Responsibilities	Collaborators
• Pre-create inactive copies of a prefab • Hand out and take back items • Recycle the oldest item when full	–

Layers

Static

Responsibilities	Collaborators
• Know the physics layer numbers and masks • Decide which layers collide (one source of truth)	–

PlayerStats

ScriptableObject

Responsibilities	Collaborators
• Know movement tuning (speed, acceleration, air control) • Know jump tuning (height, time to apex, coyote time, buffer) • Compute gravity and jump velocity from height and time • Know roll tuning (duration, speed, cooldown) • Know sword tuning (damage, timings, hitbox, lunge) • Know health tuning (hearts, invulnerability, knockback) • Compute jump reach for the level checker	–

EnemyData

ScriptableObject (Type Object)

Responsibilities	Collaborators
• Know identity (name, layout symbol, prefab, sprite, animation) • Know stats (health, speeds, sight, patrol) • Know attack style, timings, hitbox and knockback • Know contact damage • Know immunities (Brute ignores Sword) • Know despawn delay and death sound	CharacterArt

BossData

ScriptableObject

Responsibilities	Collaborators
• Know max health and phase-two threshold • Know per-phase speed, telegraph and recovery times • Know charge and summon settings • Know the summoned enemy type	EnemyData

LevelData
ScriptableObject

Responsibilities	Collaborators
• Know index, name, subtitle and boss-level flag • Know the text layout, hints and moving-platform tuning • Know tiles (with variants), backgrounds and tints • Know decorations and backdrop scenery • Know disaster type, front sprites and speed profile • Know fire timings, ambient debris, music and par time	–

LevelCatalog
ScriptableObject

Responsibilities	Collaborators
• Know the ordered list of levels • Look up a level by index; report count and last level • Find the EnemyData for a layout symbol • Know the boss and player stats	LevelData EnemyData BossData PlayerStats

LevelPrefabs
ScriptableObject

Responsibilities	Collaborators
• Know every prefab a level is built from (knight, props, hazards, boss, princess, front)	–

SpawnEntry
Struct

Responsibilities	Collaborators
• Know an enemy type and where it spawns	EnemyData

AudioLibrary
ScriptableObject

Responsibilities	Collaborators
• Know every sound and music track by id • Know volume, pitch variance and repeat limit per sound	–

CharacterArt / SpriteClip
ScriptableObject + serializable clip

Responsibilities	Collaborators
• Know a character's named animation clips • Know each clip's frames, speed and looping	–

InputReader
Only class that touches the Input System

Responsibilities	Collaborators
• Know the horizontal move axis and jump-held • Raise Jump, Roll, Attack, Pause and Cancel events • Switch between Gameplay and UI action maps	InputBindings

InputBindings
Static

Responsibilities	Collaborators
• Hold the action maps and default key bindings as JSON (incl. J, 0, click to attack)	–

IDamageable
Interface. Implemented by: Health

Responsibilities	Collaborators
• Accept damage and report whether it was applied • Report whether it is alive	DamageInfo

DamageInfo
Struct

Responsibilities	Collaborators
• Know amount, damage type, source, hit point • Know knockback direction and force, lethal flag	–

HealthModel
Plain C# class

Responsibilities	Collaborators
• Know current and max health • Apply damage honoring invulnerability, immunity and lethal bypass • Heal up to the maximum • Run timed and manual invulnerability	DamageInfo

Health
Implements: IDamageable

Responsibilities	Collaborators
• Configure max health and immunities • Raise Changed, Damaged, Died and Blocked events • Start invulnerability windows; kill outright	HealthModel DamageInfo

Hitbox
Component

Responsibilities	Collaborators
• Know damage, type, knockback, lethal flag and target layers • Open and close for an attack window • Hit each target once per opening • Report every hit (applied or blocked)	IDamageable DamageInfo

ContactDamage
Component

Responsibilities	Collaborators
• Hurt the knight on touch, limited by his invulnerability	IDamageable DamageInfo

PlayerController
Facade for the knight

Responsibilities	Collaborators
• Wire sibling components and apply stats • Own and tick the player state machine • Latch buffered input (jump, roll, attack) • Accept scripted movement for cut-scenes and tests • Enter Hurt / Dead state on damage / death • Relay health and death as global events	PlayerMovement PlayerCombat Health PlayerStats StateMachine PlayerState InputReader GameEvents

PlayerMovement
Component

Responsibilities	Collaborators
• Accelerate and decelerate horizontally • Apply designed gravity; jump with coyote time and jump cut • Roll with cooldown, passing through enemies • Detect ground and landing; ride moving platforms • Apply knockback, lunge and teleport	PlayerStats MovingPlatform GameEvents

PlayerCombat
Component

Responsibilities	Collaborators
• Start a swing when cooldown allows • Run wind-up, active and recovery phases • Open the sword hitbox for active frames only • Count kills; trigger hit stop, shake and sparks	Hitbox PlayerStats GameEvents

PlayerState

Abstract. Implements: IState. Subclasses: Grounded, Airborne, Rolling, Attacking, Hurt, Dead

Responsibilities	Collaborators
• Know the owning PlayerController • Provide empty state hooks • Request state changes; start attacks from free states	PlayerController StateMachine

PlayerGroundedState

Superclass: PlayerState

Responsibilities	Collaborators
• Apply horizontal input and footsteps • Go airborne on jump or ground lost • Go rolling / attacking when ready	PlayerMovement PlayerCombat PlayerAirborneState PlayerRollingState PlayerAttackingState

PlayerAirborneState

Superclass: PlayerState

Responsibilities	Collaborators
• Apply air control • Allow coyote jumps and air attacks • Return to grounded on landing	PlayerMovement PlayerAttackingState PlayerGroundedState

PlayerRollingState

Superclass: PlayerState

Responsibilities	Collaborators
• On enter: start roll and invulnerability • On exit: restore layer, start cooldown • Return to grounded / airborne	PlayerMovement Health

PlayerAttackingState

Superclass: PlayerState

Responsibilities	Collaborators
• On enter: begin swing, lock facing, lunge • Allow slowed movement • Return when the swing ends	PlayerCombat PlayerMovement

PlayerHurtState

Superclass: PlayerState

Responsibilities	Collaborators
• Cancel the swing, apply knockback • Block input during the stun, then return	PlayerMovement PlayerCombat

PlayerDeadState

Superclass: PlayerState

Responsibilities	Collaborators
• Lock input, stop the knight • Switch to the corpse layer	PlayerMovement

EnemyAI

Component (one class for every enemy type)

Responsibilities	Collaborators
• Initialize components from EnemyData • Own the enemy state machine • Know spawn anchor • React to hits (stun), blocked hits (flinch) and death • Raise kill events; despawn	EnemyData Health EnemyMotor EnemyPerception Hitbox ContactDamage StateMachine EnemyState GameEvents

EnemyMotor

Component. Also used by the boss

Responsibilities	Collaborators
• Move horizontally and face travel direction • Detect walls and ledges ahead • Apply knockback with resistance • Freeze on death	–

EnemyPerception

Component

Responsibilities	Collaborators
• Detect the knight in range (front and rear) with line of sight • Know target position, distance and direction • Remember the target briefly after losing it	PlayerController

EnemyState

Abstract. Implements: IState. Subclasses: Patrol, Chase, Attack, Stunned, Dead

Responsibilities	Collaborators
• Know the owning EnemyAI • Provide hooks, a change helper and Resume (chase or patrol)	EnemyAI StateMachine

EnemyPatrolState

Superclass: EnemyState

Responsibilities	Collaborators
• Start walking toward the knight's side • Patrol within radius, pausing at the ends • Switch to chase when the knight is seen	EnemyMotor EnemyPerception EnemyChaseState

EnemyChaseState

Superclass: EnemyState

Responsibilities	Collaborators
• Move toward the knight at chase speed • Stop at ledges • Attack when in range and off cooldown	EnemyMotor EnemyPerception EnemyAttackState

EnemyAttackState

Superclass: EnemyState

Responsibilities	Collaborators
• Telegraph, strike, recover • Swipe / Slam in place or Charge forward (Runner) • Open the attack hitbox for the strike	EnemyMotor Hitbox EnemyData EnemyChaseState

EnemyStunnedState

Superclass: EnemyState

Responsibilities	Collaborators
• Flinch briefly after a hit or a blocked hit • Turn to face an attacker behind it	EnemyMotor EnemyPerception

EnemyDeadState

Superclass: EnemyState

Responsibilities	Collaborators
• Disable contact damage and hitbox • Fade out, then despawn	EnemyAI Hitbox

EnemyFactory

Factory

Responsibilities	Collaborators
• Create an enemy from EnemyData (only place enemies are made) • Stream the spawn table in ahead of the knight • Despawn enemies behind the front • Spawn boss summons; create the boss • Know live enemies	EnemyData EnemyAI SpawnEntry BossController

BossController
Component

Responsibilities	Collaborators
• Initialize components from BossData	BossData
• Start the fight when the knight enters the arena	Health
• Lock and release arena walls	EnemyMotor
• Enter phase two below the health threshold	EnemyPerception
• Summon Shamblers	Hitbox
• Raise boss intro, health and defeated events	StateMachine
	BossState
	EnemyFactory
	ArenaWall
	GameEvents

BossState
Abstract. Implements: IState. Subclasses: Intro, PhaseOne, PhaseTwo, Defeated

Responsibilities	Collaborators
• Know the owning BossController	BossController
• Provide hooks and a change helper	StateMachine

BossIntroState
Superclass: BossState

Responsibilities	Collaborators
• Freeze the boss and roar	BossController
• Go to phase one after the intro	

BossCombatState
Abstract. Superclass: BossState (Template Method)

Responsibilities	Collaborators
• Run the fight loop: approach, telegraph, swing, recover	EnemyMotor
	Hitbox
• Summon on a timer	BossController
• Charge across the arena when allowed	
• Let subclasses supply the numbers	

BossPhaseOneState
Superclass: BossCombatState

Responsibilities	Collaborators
• Slow telegraphed swings and regular summons	BossData

BossPhaseTwoState
Superclass: BossCombatState

Responsibilities	Collaborators
• Faster swings, shorter recovery, charge attack, fewer summons	BossData

BossDefeatedState
Superclass: BossState

Responsibilities	Collaborators
• Disable hitboxes and collisions	BossController
• Fade out, release the arena, announce defeat	GameEvents

LevelController
One per level scene

Responsibilities	Collaborators
• Load level data for the current index	GameManager
• Spawn the knight at the start or last checkpoint	LevelCatalog
• Set camera bounds and backdrop	LevelPrefabs
• Place and configure the disaster front	LevelBuilder
• Stream enemies; create the boss on level 8	EnemyFactory
• Report completion when the exit is reached	DisasterFront
• Pause the front for the boss; clear summons on defeat	CameraFollow
	ParallaxBackground
	EndingController
	AmbientDebris

LevelBuilder
Component

Responsibilities	Collaborators
• Turn the text layout into tilemaps (with tile variants) and colliders	LevelLayout
	LevelData
• Place props, hazards, checkpoints, exit, signs	LevelPrefabs
• Build the spawn table	BuiltLevel
• Place backdrop scenery on flat ground	
• Set level walls, kill zone and bounds	

LevelLayout
Plain C# class

Responsibilities	Collaborators
• Parse the text grid (one character per tile)	–
• Know solid and one-way cells and markers	
• Validate a level's rules	

LevelReachability
Plain C# class

Responsibilities	Collaborators
• Prove a level is finishable with the knight's real jump	LevelLayout
	PlayerStats
• Account for platforms and moving platforms	

BuiltLevel
Plain C# class

Responsibilities	Collaborators
• Know what the builder produced (start, checkpoints, exit, spawns, boss, princess, arena, bounds)	–

DisasterFront
Component

Responsibilities	Collaborators
• Advance each physics step	FrontModel
• Kill the knight when caught behind it	LevelData
• Publish distance to the knight	PlayerController
• Animate the wall; rumble, shake and bolts when close	Health
	GameEvents
• Pause, resume, stop at the boss arena	

FrontModel
Plain C# class

Responsibilities	Collaborators
• Know position, elapsed time, paused flag	–
• Ramp speed over time with a cap	
• Add capped catch-up when far behind	
• Never move backwards; respect a stop line	

Checkpoint
Component

Responsibilities	Collaborators
• Know its index and spawn point	PlayerController
• Activate on first contact; raise event	GameEvents
• Show activated flag	

ExitGate
Component

Responsibilities	Collaborators
• Open as the knight approaches	PlayerController
• Signal 'reached' once	GameEvents

KillZone
Component

Responsibilities	Collaborators
• Apply lethal fall damage to anything entering	IDamageable DamageInfo

HeartPickup
Component

Responsibilities	Collaborators
• Heal one heart only if the knight is hurt • Raise pickup event and disappear	Health GameEvents

MovingPlatform
Component

Responsibilities	Collaborators
• Travel back and forth with pauses • Know its velocity so the knight rides it • Size itself to its width	–

ArenaWall
Component

Responsibilities	Collaborators
• Rise to lock the arena; sink to release it	–

TutorialSign
Component

Responsibilities	Collaborators
• Show a hint once when the knight passes	GameEvents

EndingController
Castle level only

Responsibilities	Collaborators
• On boss defeat, take control of the knight • Open the cage; walk the knight to the princess • Play the reunion, then tell GameManager the game is finished	GameEvents PlayerController GameManager

ParallaxBackground
Component

Responsibilities	Collaborators
• Show sky, far and near layers scaled to height • Scroll each layer slower than the camera	LevelData

Hazard
Abstract (Template Method). Subclasses: FirePatch, FallingDebris, Spikes

Responsibilities	Collaborators
• Own a hitbox that hurts the knight and enemies • Run the fixed cycle: telegraph, activate, stay active, deactivate, rest	Hitbox DamageInfo

FirePatch
Superclass: Hazard

Responsibilities	Collaborators
• Glow as a warning, burn, go out on a timer • Re-damage occupants at intervals (lure Brutes)	Hitbox

FallingDebris
Superclass: Hazard

Responsibilities	Collaborators
• Show a shadow, then drop a rock • Hit once on impact, then break • Tell a pool when done	Hitbox PlayerController

Spikes
Superclass: Hazard

Responsibilities	Collaborators
• Stay active; re-hit at intervals	Hitbox

AmbientDebris
Component

Responsibilities	Collaborators
• Drop pooled rocks ahead of the knight at intervals	ObjectPool FallingDebris LevelLayout

CameraFollow
Component

Responsibilities	Collaborators
• Follow the knight with smoothing and look-ahead • Clamp to level bounds • Shake on request	PlayerController GameEvents

AudioManager
Singleton

Responsibilities	Collaborators
• Play music with cross-fades and sound effects from a voice pool • Turn game events into sounds • Pick music by game state and level • Apply saved volumes	AudioLibrary GameEvents GameManager

SpriteAnimator
Component

Responsibilities	Collaborators
• Play named clips frame by frame • Show a clip at a given progress (sword swing)	CharacterArt

PlayerAnimator / EnemyAnimator / BossAnimator
Components

Responsibilities	Collaborators
• Choose the clip for the current state • Add squash, tilt, telegraph glow, hit flash and blink	SpriteAnimator PlayerController EnemyAI BossController

EffectPlayer
Component

Responsibilities	Collaborators
• Play dust, sparks, slashes and debris on request • Reuse effects through one pool per type	ObjectPool PooledParticle GameEvents

UIFlowController
Component

Responsibilities	Collaborators
• Listen for game state changes • Show the screens for each state • Handle Back / Cancel and sub-menus • Point the UI input module at the UI action map	GameEvents InputReader UIPanel

UIPanel
Abstract. Subclasses: MainMenu, LevelSelect, Settings, HUD, Pause, GameOver, LevelComplete, Credits

Responsibilities	Collaborators
• Build its widgets once from code • Show / hide and select the first button	UIFactory UIStyle UIFlowController

UIFactory / UIStyle

Static helpers + ScriptableObject

Responsibilities	Collaborators
• Create styled text, buttons, sliders and panels • Know fonts, sprites and colours	–

HUDController

Superclass: UIPanel

Responsibilities	Collaborators
• Show level name and timer • Show the front distance meter • Show hints and the boss health bar	GameEvents HeartDisplay GameManager

HeartDisplay

Component

Responsibilities	Collaborators
• Render max and filled hearts • Pop hearts that are lost or gained	GameEvents

GameOverPanel

Superclass: UIPanel

Responsibilities	Collaborators
• Show the cause of death • Offer Retry, Restart, Main menu	GameManager

LevelCompletePanel

Superclass: UIPanel

Responsibilities	Collaborators
• Show time, best time, hearts and kills • Offer Next, Replay, Main menu	GameManager LevelResult

Main / LevelSelect / Settings / Pause / Credits

Superclass: UIPanel

Responsibilities	Collaborators
• Start or continue the game; pick unlocked levels • Set volumes, fullscreen, reset progress • Resume, retry, restart; show the ending credits	GameManager SaveSystem AudioManager

ScreenFader

Component

Responsibilities	Collaborators
• Fade to and from black between scenes (unscaled time)	–